

Facultad de Ciencias UNAM

Tarea 3: Propiedades de Orden Asintótico y Recurrencias

Profesora: Virginia Teodosio Procopio

Ayudantes: Leonardo Gallo Guerrero
José Manuel Madrigal Ramírez

Entrega: 3 de Octubre de 2026

Instrucciones

No olvides seguir las instrucciones para la entrega de tareas que puedes encontrar tanto en nuestro repositorio de Github como en el sitio del curso.

Propiedades de orden asintótico

Considerando la siguiente:

Definición (O-grande) Sean f y g funciones de complejidad, decimos que f está en el orden de g $f \in O(g)$, y g representa una cota asintótica superior para f si y sólo si $\exists c \in \mathbb{R}^+$ y $\exists n_0 \in \mathbb{N}$ tales que $\forall n \in \mathbb{N}$ y $n > n_0$ tenemos que $f(n) \leq cg(n)$.

Realiza los siguientes **ejercicios**:

1. Sea $T(n) = 2n^2 + 6n$, demuestra que $T(n) \in O(n^3)$. (0.75 pto.)
2. Sea $T(n) = 3\sqrt{n} + 15n^2$ demuestra que $T(n) \in O(n^2)$. (0.75 pto.)
3. (Transitividad de O) Sean f , g y h tres funciones de complejidad tales que $f \in O(g)$ y $g \in O(h)$, demuestra que $f \in O(h)$. (1 pto.)

Recurrencias

Pistas

Recuerda que: Una relación de recurrencia expresa una función en términos de valores anteriores de esa misma función. Para determinarla completamente, **es necesario especificar sus casos base**.

Cuando queremos definir una función $T(n)$ que cuantifique el tiempo de ejecución de un algoritmo recursivo, construimos una recurrencia considerando, entre otros elementos:

- el costo de los casos base,
- el número de llamadas recursivas,
- el tamaño de cada subproblema,
- el trabajo realizado fuera de las llamadas recursivas.

Al construir $T(n)$ es necesario establecer qué modelo de costo estamos utilizando: podemos contar todas las operaciones elementales del algoritmo o concentrarnos en una operación esencial, pero debemos mantener la misma convención durante todo el análisis.

Ejercicios

En cada uno de los siguientes algoritmos, hemos escrito una propuesta de recurrencia que define su función de complejidad y en cada caso debes:

- a) Escribir si la recurrencia planteada es adecuada para el algoritmo en cuestión, en caso de no serlo, plantea una expresión de recurrencia correcta. No olvides justificar tu respuesta. (0.75 pto. c/u)
- b) Resuelve la recurrencia mediante sustitución y encuentra una expresión sin recursión para la función de complejidad $T(n)$. Puedes utilizar como ayuda los ejercicios que vimos en clase o las notas que se encuentran en el sitio del curso. (1 pto. c/u)
- c) Encuentra una función $g(n)$ que describa una cota superior asintótica para la función de complejidad $T(n)$ y demuestra que $T \in O(g)$. (0.75 pto. c/u)

Algoritmo 1 SUMA(n)

Precondiciones: n es un entero, $n \geq 0$

```
1: if  $n = 0$  then
2:   return 0
3: else
4:   return  $n + \text{SUMA}(n - 1)$ 
5: end if
```

Expresión de recurrencia candidata a definir el tiempo de ejecución para SUMA(n):

$$T(n) = T(n - 1) + c \tag{1}$$

Algoritmo 2 DIGITOS(n)

Precondiciones: n es un entero, $n \geq 1$

```
1: if  $n < 10$  then  
2:   return 1  
3: else  
4:   return  $1 + \text{DIGITOS}(\lfloor n/10 \rfloor)$   
5: end if
```

Expresión de recurrencia candidata a definir el tiempo de ejecución para DIGITOS(n):

$$T(n) = \begin{cases} c & n \leq 9 \\ T(\lfloor n/10 \rfloor) + k & n > 9 \end{cases} \quad (2)$$

Algoritmo 3 POTENCIA(x, n)

Precondiciones: x es un número y n es un entero, $n \geq 0$

```
1: if  $n = 0$  then  
2:   return 1  
3: else  
4:   return  $x \times \text{POTENCIA}(x, n - 1)$   
5: end if
```

Expresión de recurrencia candidata a definir el tiempo de ejecución para POTENCIA(x, n):

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n) - 1 & n > 0 \end{cases} \quad (3)$$

